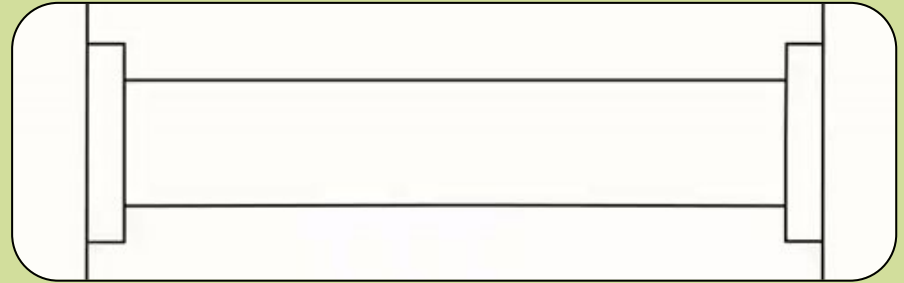


PHYS 246

Finals Project

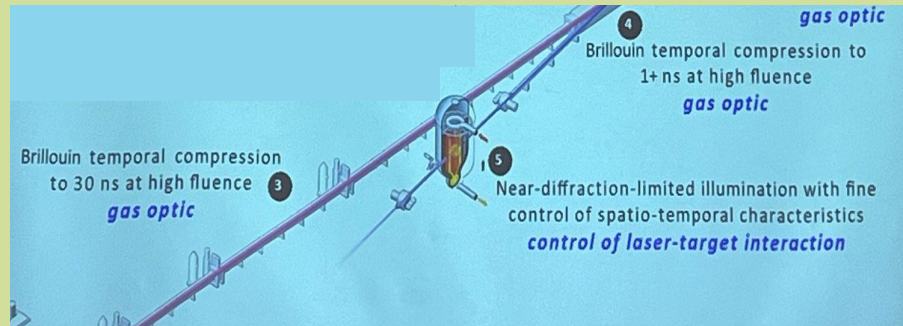
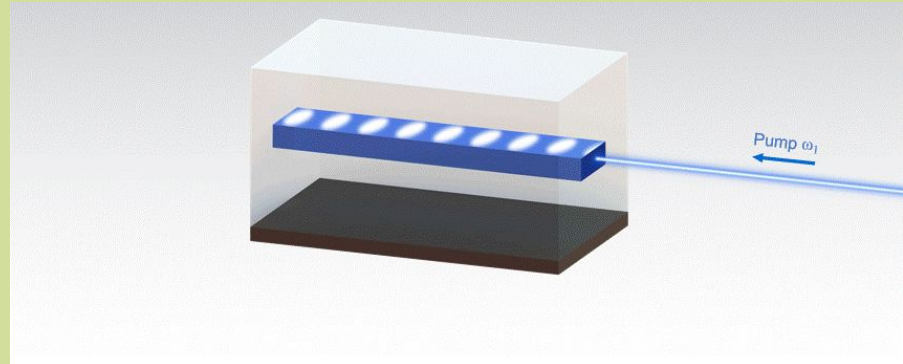
Stimulated Brillouin
Scattering in Optical
Fibers



Shashwat Joglekar, Mike Wen

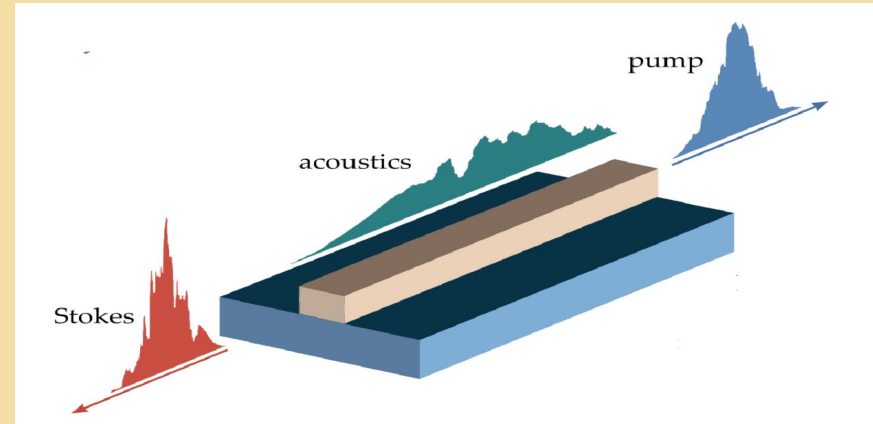
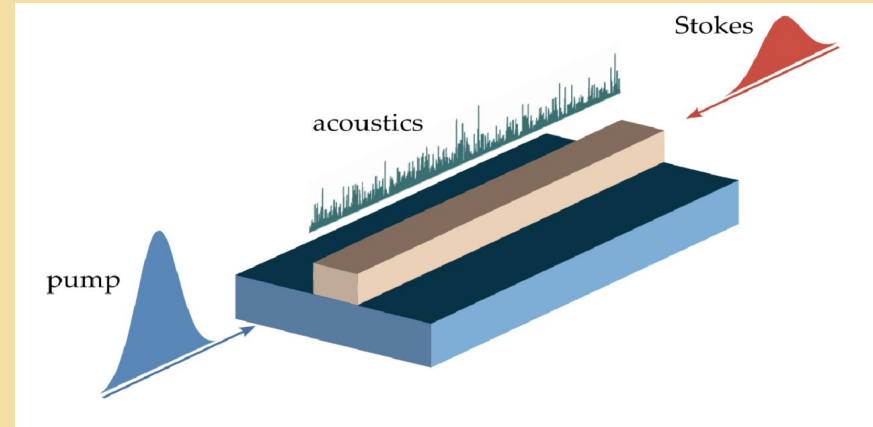
SBS and its Applications

1. Electrostriction and “Stored Light” via sound waves
 - a. “Stokes” + “Anti-Stokes”
 - b. Resultant “Brillouin shift” Ω
2. Filtering and amplification
 - a. Compression of a high-power pump pulse
 - b. Amplifying short seed pulse
3. Short data buffers in optical fibers
4. Compressing ~ 1 petawatt / 1 MJ-scale fusion driver pulses by $\sim 1000\times$ in time domain



Objectives

1. Replicate the basic SBS behavior in the following paper:
Oscar A. Nieves, Matthew D. Arnold, Michael J. Steel, Mikołaj K. Schmidt, and Christopher G. Poulton, "Numerical simulation of noise in pulsed Brillouin scattering," J. Opt. Soc. Am. B 38, 2343-2352 (2021)
2. Understand effects of gain coefficients on coupling and noise
3. Produce numerically-stable and reasonably physical results



Physical Parameters

1. Gain parameter g_0
2. Brillouin shift Ω , ~10GHz
 - a. Due to propagation of sound wave
 - b. Can be replaced by off-axis incoming waves
3. Acoustic and source noise D, Φ
4. Cavity length - longer = better!
 - a. Longer time and length of pump/stokes interaction
 - b. Longer acoustic wave
5. v, v_a wave propagation velocities

```
# Setting initial variables
params_groundtruth = {
    # Physical properties
    'v': 3e8 / 2.44, # Speed of light in our medium, m / s
    'v_a': 2500, # Speed of sound in waveguide medium, m / s
    'temp': 300, # Temperature in Kelvin
    'alpha': 0, # Optical decay rate due to inelastic processes, set to 0 here since we want to focus on simulating SRS gain
    'alpha_ac': 7.54e4, # Acoustic attenuation in m^-1
    'k_B': 1.38e-23, # Boltzmann constant in J / K

    'linewidth_Brill': 30e3, # Brillouin linewidth, effective maximum possible difference of frequency in the signal from its central frequency
    'shift_Brill': 7.7e9 * 2 * np.pi, # Hz, Brillouin shift between input and output carrier frequencies. The input is downshifted by this
    'Brillouin_gain': 423e2, # m^(-1) W^(-1), describes the coupling between optical and acoustic waves; the longer the interaction distance and the greater the input power, the greater the output

    'wavelength': 1550e-9, # Meters
    'FWHM_stokes': 1e-9, # Temporal half-magnitude pulse widths for pump pulse power, in seconds
    # FWHM for pump pulse, but we'll start at 0.2 ns and increment up by 0.5ns
    'p_max_pump': 1, # Watts. This is pretty high for a laser!
    # For peak power, we'll vary that of the stokes pulse from 0.1 mW up to 1 mW

    'linewidth_laser': 100e3, # Hz, similar to Brillouin linewidth, we'll use this to compute how the step size for the phase noise from our laser sources

    # Numerical simulation properties
    'L': 2, # Length of waveguide cavity in meters
    'N_x': 1001
}
```

Parameter	Value
Waveguide length L	50 cm
Waveguide temperature T	300 K
Refractive index n	2.44
Acoustic velocity v_a	2500 m/s
Brillouin linewidth $\Delta\nu_B$	30 MHz
Brillouin shift $\Omega/2\pi$	7.7 GHz
Brillouin gain parameter g_0	$423 \text{ m}^{-1} \text{ W}^{-1}$
Optical wavelength λ	1550 nm
Laser linewidth $\Delta\nu_L$	100 kHz
Peak pump power	1 W
Peak Stokes power	0–1 mW
Simulation time t_f	up to 80 ns
Pump pulse FWHM	0.5–5 ns
Stokes pulse FWHM	1 ns
Grid size (space) N_x	1001
Grid size (time) N_t	2601
Step-size Δt	4.07 ps

Simulation Parameters

1. Time steps and t_n
 - a. Not as crucial as for real fluid sims because spatial “spreading” of envelope fields is minimal
2. Space partitioning and z_j
 - a. Determined by Δt to simplify propagation calculations later
3. Assumption of no optical decay
4. Assumption of no acoustic wave propagation
 - a. v / v_a approximately $1e5$ - $1e6$

```
# Setting initial variables
params_groundtruth = {
    # Physical properties
    'v': 3e8 / 2.44, # Speed of light in our medium, m / s
    'v_a': 2500, # Speed of sound in waveguide medium, m / s
    'temp': 300, # Temperature in Kelvin
    'alpha': 0, # Optical decay rate due to inelastic processes, set to 0 here since we want to focus on simulating SRS gain
    'alpha_ac': 7.54e4, # Acoustic attenuation in m^-1
    'k_B': 1.38e-23, # Boltzmann constant in J / K

    'linewidth_Brill': 30e3, # Brillouin linewidth, effective maximum possible difference of frequency in the signal from its central frequency
    'shift_Brill': 7.7e9 * 2 * np.pi, # Hz, Brillouin shift between input and output carrier frequencies. The input is downshifted by this
    'Brillouin_gain': 423e2, # m^(-1) W^(-1), describes the coupling between optical and acoustic waves; the longer the interaction distance and the greater the input power, the greater the output

    'wavelength': 1550e-9, # Meters
    'FWHM_stokes': 1e-9, # Temporal half-magnitude pulse widths for pump pulse power, in seconds
    # FWHM for pump pulse, but we'll start at 0.2 ns and increment up by 0.5ns
    'pump_pulse': 1, # Watts. This is pretty high for a laser!
    # For peak power, we'll vary that of the Stokes pulse from 0.1 mW up to 1 mW

    'linewidth_Laser': 100e3, # Hz, similar to Brillouin linewidth, we'll use this to compute how the step size for the phase noise from our laser sources

    # Numerical simulation properties
    'L': 2, # Length of waveguide cavity in meters
    'N_x': 1001
}
```

Parameter

Value

Waveguide length L	50 cm
Waveguide temperature T	300 K
Refractive index n	2.44
Acoustic velocity v_a	2500 m/s
Brillouin linewidth $\Delta \nu_B$	30 MHz
Brillouin shift $\Omega/2\pi$	7.7 GHz
Brillouin gain parameter g_0	$423 \text{ m}^{-1} \text{W}^{-1}$
Optical wavelength λ	1550 nm
Laser linewidth $\Delta \nu_L$	100 kHz
Peak pump power	1 W
Peak Stokes power	0–1 mW
Simulation time t_f	up to 80 ns
Pump pulse FWHM	0.5–5 ns
Stokes pulse FWHM	1 ns
Grid size (space) N_x	1001
Grid size (time) N_t	2601
Step-size Δt	4.07 ps

Introducing Noise

Laser Pulse Noise $\phi(t)$

$\phi(t_n)$ represents the noise introduced by the source laser pulse (hence why it is only dependent on the time. We will use this to set the envelope fields at the boundaries of the dielectric waveguide). Both the aforementioned functions are continuous with respects to time (as discontinuity with respects to t wouldn't really be physical here).

```
## Getting D_0(z_j) values
T = 20.58607e-9 # Total simulation time used from paper, times 2 for longer cavity length
dz, zs, dt, t_n = partition_space_time(params_groundtruth, T) # Partition the space

# Calculating our variances for the source and waveguide noise functions, we'll of course partition our space first
params_groundtruth['sigma'] = params_groundtruth['temp'] * params_groundtruth['alpha_ac'] * params_groundtruth['kb'] # Variance used for waveguide noise function
params_groundtruth['var'] = (params_groundtruth['v_a'] * params_groundtruth['sigma']) / (params_groundtruth['alpha_ac'] * dz) # Variance used for source noise function

# Acoustic decay rate and the time-wise step size for D(z_j, t_n) are also determined
params_groundtruth['Gamma'] = params_groundtruth['v_a'] * params_groundtruth['alpha_ac']
def gamma(params, dz, dt):
    return params_groundtruth['v_a'] * np.sqrt((params_groundtruth['sigma'] * (1 - np.exp(- params_groundtruth['Gamma'] * dt)))/(2 * params_groundtruth['Gamma'] * dz))

def generateNoiseFunctions(params, dz, zs, dt, t_n):
    # D_0 is sampled at each of our z_j's
    D_0 = np.zeros_like(zs, dtype = complex)
    for i in range(len(zs)):
        # Normalized random-phase Gaussian noise for initial thermal white noise
        D_0[i] = complex(np.random.normal(0, np.sqrt(params_groundtruth['var'])), np.random.normal(0, np.sqrt(params_groundtruth['var']))) / np.sqrt(2)

    # Phi_{1, 2} is calculated for all time. It starts as 0 and takes random Gaussian walks at each time step
    # We'll also time-evolve our D_0 noise function
    phi = np.zeros((2, len(t_n)), dtype = complex)
    D = np.zeros((len(t_n), len(zs)), dtype = complex)
    D[0] = D_0

    thermal_noise_step_scale = gamma(params, dz, dt)
    thermal_noise_decay_factor = np.exp(- params_groundtruth['Gamma'] * dt / 2)
    phase_noise_step_scale = np.sqrt(2 * np.pi * params_groundtruth['Linewidth_Laser'] * dt)

    for i in range(1, len(t_n)):
        # Also print our progress
        # if i % 100 == 0:
        #     print(f'Progress: {i / len(t_n) * 100}%')

        for j in range(len(zs)):
            D[i][j] = thermal_noise_decay_factor * D[i - 1][j] + thermal_noise_step_scale * complex(np.random.normal(0, 1), np.random.normal(0, 1))

        for k in range(2):
            phi[k][i] = phi[k][i - 1] + phase_noise_step_scale * np.random.normal(0, 1)

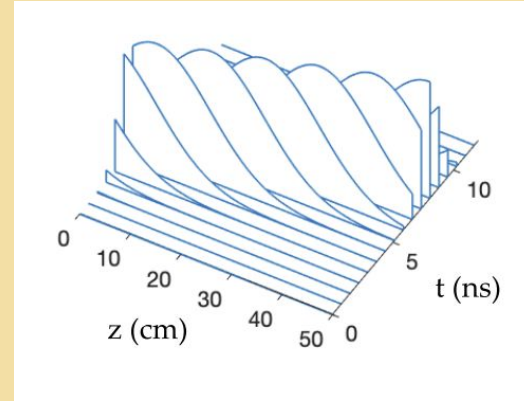
    return D_0, D, phi

# Let's run it with our parameters
D_0, D, phi = generateNoiseFunctions(params_groundtruth, dz, zs, dt, t_n)
```

Envelope Fields

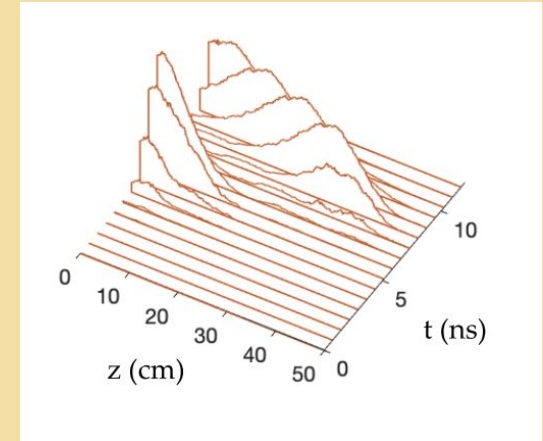
$$a_{(1,2)}(z, t)$$

$a_{1,2}(z_j, t_n)$ represent the discretely-sampled "envelope" electromagnetic fields (real part $\mathbf{E}$ and imaginary part $\mathbf{B}$), ie: the function modulating the carrier harmonics (which have the form of d'Alembert's solutions to Maxwell's equations) of the either field at a given location z_j and time t_n .



Pump

Stokes

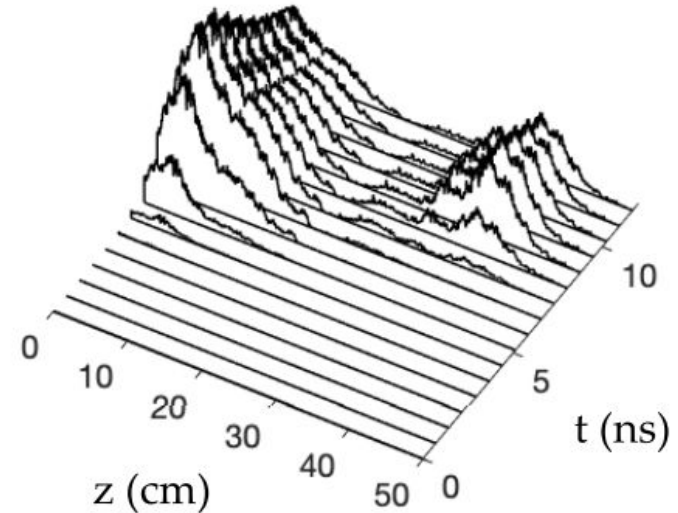


Acoustic Field

$$b(z, t)$$

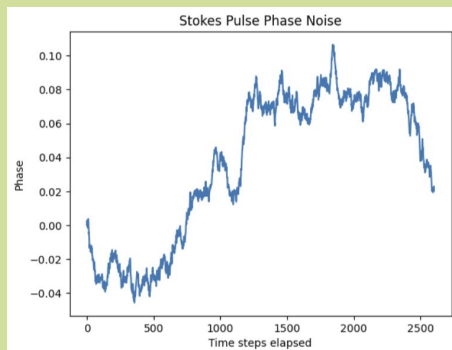
$b(z_j, t_n)$ represent the discretely-sampled "envelope" acoustic field

This will be the key mediator between the superimposed a_1 and a_2 optical envelope fields, as we'll see later

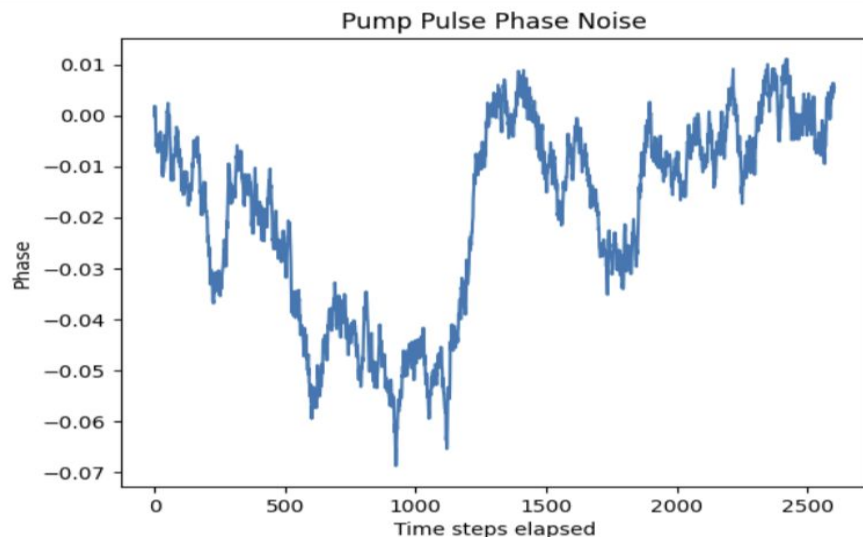


Setting Things Up

1. Setting up cavity thermal noise
2. Setting up laser phase noise (phase is coherent at a given time)
3. Both are random Gaussian walks
4. Variance is determined by gain, temperature, v_a , etc...



- 1: Compute $D(z_j, t_n)$ for all t_n
- 2: Compute $\phi_{1,2}(t_n)$ for all t_n
- 3: Set $a_{1,2} = 0$ inside $z \in [0, L]$



Looping t_n 's

1. Inserting noisy boundary conditions (independent incoming laser sources)
2. Moving optical fields a_1, a_2
3. Interaction integral I_{12}
 - a. Boiled-down version of acoustic field b generation and Stokes / anti-Stokes
 - b. Decay old "interaction"
 - c. Add on two terms (use complex conjugates)
4. Use I_{12} to alter a_1, a_2 magnitudes
 - a. Conservation of energy

4: **for** $n = 1$ to $N_t - 1$ **do** $\triangleright N_t = \text{size of time grid}$
 5: Insert noisy boundary conditions in $a_{1,2}$ at t_n
 6: Shift optical fields $a_{1,2}$ in space by Δz
 7: Compute interaction integral $I_{1,2}(z_j, t_n)$
 8: Compute $a_{1,2}(z_j, t_{n+1})$ from $a_{1,2}(z_j, t_n)$
 9: Compute $b(z_j, t_{n+1})$
end for

```
# Let's now evolve the fields over time. Electing against storing the envelope field for all time and z's as that is computationally costly
# First, some helper functions that given A1 or A2, perform each of steps 5-9 specified in the paper
def insertBoundaryConditions(A1, A2, i, A_1_0, A_2_0):
    # i indexes through a_1_0, a_2_0
    # We'll just insert the boundary conditions and return A
    A1[i] = A_1_0[i]
    A2[i] = A_2_0[i]
    return A1, A2

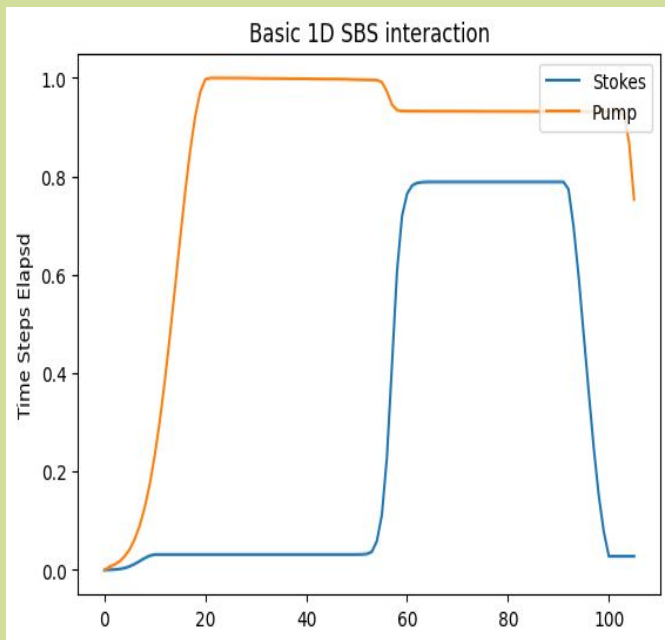
def shiftOpticalFields(A1, A2):
    # The dz is implicit here, so we just use roll
    # A1 is the pump pulse, and A2 is the stoke's pulse
    # These two are separate except for the acoustic interactions, due to linear dielectric properties and superposition
    A1 = np.roll(A1, 1)
    A2 = np.roll(A2, -1)
    return A1, A2

def stepInteractionIntegral(I, params, A1, A1_past, A2, A2_past, i, delta_t):
    # We take the input to be an array of complex 0s
    # Factoring out the exponential from equation (26) in the paper, we see that this is simply multiplying on the result of the decay factor Gamma
    # The decay is also meant to suppress the old I to make sure nothing blows up numerically
    # Expect i to be at least 1
    lead_coeff = np.exp(-params['Gamma'] * delta_t / 2)
    # Everything is in np arrays, so we can just do element-wise multiplication
    # The input A1's and A1_past's are required for numerical stability (the other reason we have the i in the exponential)
    first_term = np.conj(A1_past) * A2_past * np.exp(-params['Gamma'] * delta_t / 2)
    second_term = np.conj(A1) * A2
    # Update I and return
    I_new = (delta_t / 2) * ((lead_coeff * I) + first_term + second_term)
    return I_new

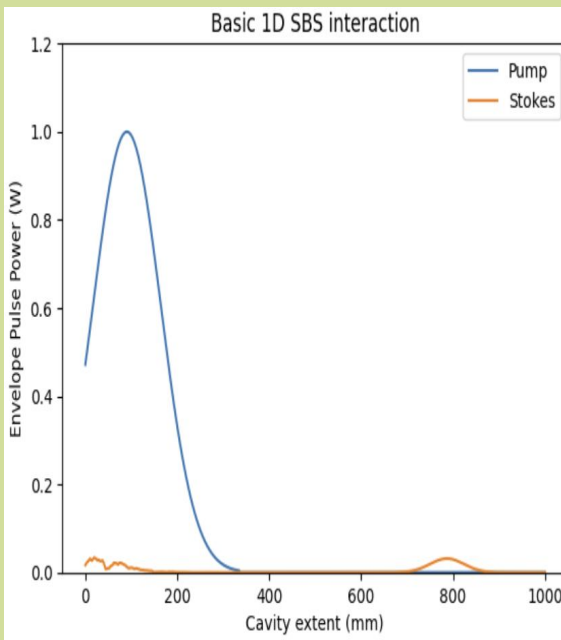
# Interaction integral (z_n indexed arrays) starts as an array of complex 0s
def step_both_fields(I, A1, A2, params, delta_t, D_n):
    # Note that the noise at time index t_n is also inputted (this is a z_n indexed array)
    decay_coeff = (1 - params['v']) * params['alpha'] * delta_t / 2
    A1_new = (decay_coeff * A1
              - params['v'] * delta_t * (params['g_1'] * params['Gamma'] + np.conj(I) / 4
              - 1) * params['omega_1'] + params['Q_1'] * np.conj(D_n)) * A2
    A2_new = (decay_coeff * A2
              + params['v'] * delta_t * (params['g_2'] * params['Gamma'] + I / 4
              - 1) * params['omega_2'] * params['Q_2'] * D_n) * A1
    return A1_new, A2_new

# We also want to calculate the acoustic field envelope
def get_acoustic_field(params, I, D_n, dz):
    first_term = 1) * params['v_a'] * params['Q_a'] * params['Shift_Brill'] * I
    second_term = np.sqrt(dz) * D_n
    return first_term + second_term
```

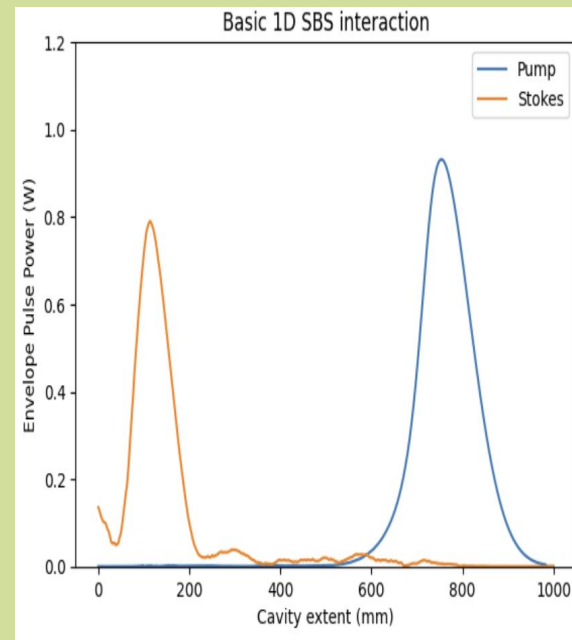
Results



Max Power vs. Time



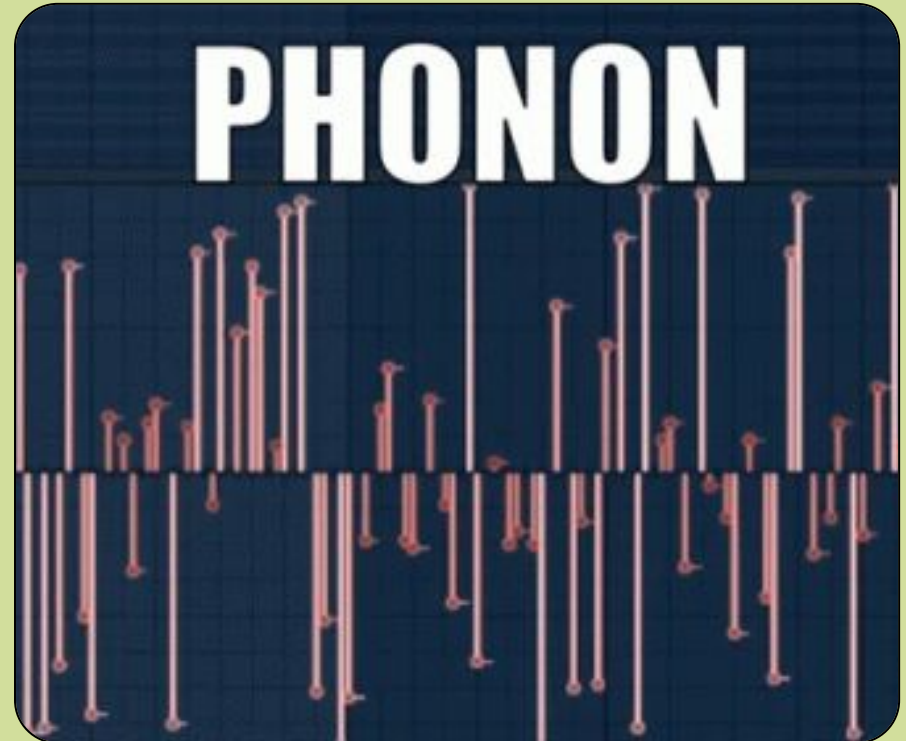
Before



After

Conclusion and Sources :)

Oscar A. Nieves, Matthew D. Arnold, Michael J. Steel, Mikołaj K. Schmidt, and Christopher G. Poulton, "Numerical simulation of noise in pulsed Brillouin scattering," J. Opt. Soc. Am. B 38, 2343-2352 (2021)



Thanks for Listening! :)